

IU Internationale Hochschule

3. Finalisierungsphase – Buchtausch-App

Vorgelegt von:

Eric Chris Taurat
Hauptstraße 38
69214 Eppelheim
E-Mail: erictaurat16@gmail.com

Matrikelnummer: 10247605
Fachsemester: 4
Studiengang: Informatik

Abgabe: 23.06.2026
Lehrkraft: Anna Androvitsanea

Buchtausch-App — Phase 3: Finalisierungsphase

1. Abstract

Die Buchtausch-App ist eine datenbankgestützte Plattform, die es Nutzern ermöglicht, physische Bücher kostenlos untereinander auszuleihen. Ziel des Projekts war die Konzeption und Implementierung eines vollständigen relationalen Datenbanksystems auf Basis von PostgreSQL. Im Mittelpunkt standen dabei sowohl die fachliche Modellierung der Anforderungen als auch die technische Umsetzung unter Berücksichtigung von Datenintegrität, Normalisierung und Performanz. Die Anwendung bildet sämtliche zentralen Geschäftsprozesse einer Buchtauschplattform ab. Dazu gehören die Registrierung und Verwaltung von Nutzern, die Erfassung und Katalogisierung von Büchern, die Verwaltung einzelner Buchexemplare, die Organisation von Ausleihvorgängen über Zeitslots, ein Bewertungssystem zwischen Nutzern sowie ein integrierter Nachrichtenaustausch. Zusätzlich werden Standortdaten gespeichert, um künftig eine geografische Suche nach verfügbaren Büchern in der Nähe zu ermöglichen.

Das entwickelte Datenbankschema umfasst insgesamt 14 Tabellen und wurde konsequent in der dritten Normalform (3NF) modelliert. Ziel war es, Redundanzen zu vermeiden, Datenkonsistenz sicherzustellen und zukünftige Erweiterungen zu erleichtern. Hierfür wurden wiederkehrende Werte wie Genres, Sprachen, Verlage und Buchzustände in eigene Lookup-Tabellen ausgelagert. Diese Vorgehensweise reduziert Datenwiederholungen und erhöht die Wartbarkeit der Datenbank.

Ein wesentlicher Fortschritt gegenüber Phase 2 bestand in der Einführung einer separaten Autorenverwaltung. Statt Autoren lediglich als Textfeld innerhalb der Büchertabelle zu speichern, wurde eine eigenständige Tabelle `authors` mit einer M:N-Beziehung über die Brückentabelle `book_authors` implementiert. Dadurch können mehrere Autoren einem Buch zugeordnet werden und gleichzeitig dieselben Autoren mehreren Werken zugewiesen werden. Diese Lösung verbessert die Normalisierung und entspricht realen Anforderungen bibliografischer Systeme.

Die Kernfunktionalitäten der Plattform werden durch mehrere relationale Verknüpfungen abgebildet. Besonders hervorzuheben sind die Tabellen `loans`, `ratings` und `messages`, welche komplexe Beziehungen zwischen verschiedenen Entitäten modellieren. Darüber hinaus wurden Trigger implementiert, um bestimmte Geschäftsregeln automatisch durchzusetzen. Beispielsweise werden überfällige Ausleihen automatisch als verspätet markiert und die Verfügbarkeit von Buchexemplaren wird bei Rückgabe oder Ausleihe automatisch aktualisiert.

Zur Optimierung der Datenbankleistung wurden zwölf Indizes definiert und mithilfe von EXPLAIN-Analysen untersucht. Die Ergebnisse zeigen, dass häufig genutzte Such- und Join-Operationen effizient ausgeführt werden können. Insgesamt entstand ein vollständig funktionsfähiges relationales Datenmodell, das sowohl die fachlichen Anforderungen erfüllt als auch technische Qualitätskriterien moderner Datenbanksysteme berücksichtigt.

2. Making of – Technische Herangehensweise

Phase 1 – Analyse und Datenmodellierung

Zu Beginn des Projekts erfolgte die Analyse der fachlichen Anforderungen. Dabei wurden die zentralen Anwendungsfälle einer Buchtauschplattform identifiziert und in ein konzeptionelles Datenmodell überführt. Mithilfe eines Entity-Relationship-Diagramms (ERD), das mit draw.io erstellt wurde, konnten die relevanten Entitäten und deren Beziehungen visualisiert werden.

Im Zentrum des Modells standen die Entitäten `users`, `books` und `book_copies`. Nutzer können mehrere Bücher besitzen, während jedes Buch wiederum in mehreren physischen Exemplaren vorhanden sein kann. Bereits in dieser frühen Entwurfsphase wurde darauf geachtet, Beziehungen möglichst präzise abzubilden und spätere Erweiterungen zu berücksichtigen.

Bestimmte Prozesse wie Ausleihen, Bewertungen oder Nachrichten betreffen mehrere Entitäten gleichzeitig und lassen sich nicht sinnvoll durch einfache Fremdschlüsselbeziehungen darstellen. Aus diesem Grund wurden die Tabellen `loans`, `ratings` und `messages` als eigenständige relationale Konstrukte modelliert.

Darüber hinaus wurden geeignete Kardinalitäten definiert und die Grundlage für eine Normalisierung bis zur dritten Normalform geschaffen. Ziel war es, Mehrfachspeicherungen derselben Informationen zu vermeiden und eine konsistente Datenhaltung sicherzustellen.

Phase 2 – Implementierung und Testdaten

Nach Abschluss der Modellierungsphase wurde das Datenbankschema in PostgreSQL implementiert. Sämtliche Tabellen wurden mithilfe von CREATE-TABLE-Anweisungen angelegt und mit geeigneten Integritätsbedingungen versehen.

Zum Einsatz kamen verschiedene Constraint-Typen:

- PRIMARY KEY zur eindeutigen Identifikation von Datensätzen
- FOREIGN KEY zur Sicherstellung referenzieller Integrität
- UNIQUE-Constraints zur Vermeidung von Duplikaten
- CHECK-Constraints zur Durchsetzung fachlicher Regeln
- DEFAULT-Werte für häufig verwendete Standardwerte

Anschließend wurden für jede Tabelle mindestens zwölf Testdatensätze erzeugt. Dabei wurde darauf geachtet, möglichst viele unterschiedliche Beziehungstypen abzudecken. Die Testdaten simulieren reale Nutzungsszenarien und ermöglichen eine umfassende Überprüfung der Datenbankfunktionalität.

Zur Qualitätssicherung wurden insgesamt 22 automatisierte Testfälle in PL/pgSQL entwickelt. Diese Tests überprüfen unter anderem:

- die Einhaltung von Fremdschlüsselbeziehungen,
- die Korrektheit von Kardinalitäten,
- die Funktionsfähigkeit von CHECK-Constraints,
- die Konsistenz von Statuswerten,

- die Existenz erforderlicher Datensätze.

Alle Tests wurden erfolgreich ausgeführt und liefern den Status „PASS“. Dadurch konnte nachgewiesen werden, dass das implementierte Datenmodell sowohl technisch korrekt als auch fachlich konsistent ist.

Phase 3 – Finalisierung und Optimierung

Im Rahmen der Finalisierungsphase wurden die Rückmeldungen aus der Bewertung von Phase 2 analysiert und gezielt umgesetzt.

Die wichtigste strukturelle Verbesserung war die Einführung einer separaten Autorenverwaltung. Die ursprüngliche Lösung mit einem einfachen Autorenfeld innerhalb der Büchertabelle wurde durch die Tabellen authors und book_authors ersetzt. Dadurch können nun beliebig viele Autoren pro Buch gespeichert werden. Gleichzeitig wird verhindert, dass Autorennamen mehrfach und potenziell inkonsistent gespeichert werden.

Ein weiterer Schwerpunkt lag auf der Automatisierung wiederkehrender Prozesse. Hierfür wurden zwei PL/pgSQL-Trigger implementiert. Der erste Trigger überprüft Ausleihen auf Überschreitung des Rückgabedatums und aktualisiert den Status automatisch auf „überfällig“. Der zweite Trigger verwaltet die Verfügbarkeit von Buchexemplaren und passt diese bei Ausleihen oder Rückgaben automatisch an. Dadurch wird die Konsistenz der Datenbank erhöht und der manuelle Verwaltungsaufwand reduziert.

Zusätzlich wurden die Testdaten erweitert. In Phase 2 war die Beziehung zwischen Buchexemplaren und Zeitslots zwar modelliert, jedoch nicht vollständig durch die Beispieldaten repräsentiert. In der Finalisierungsphase wurden mehrere Zeitslots pro Exemplar ergänzt, sodass die vorgesehene 1:N-Beziehung nun vollständig nachvollzogen werden kann.

Abschließend wurde die Performance der Datenbank untersucht. Hierfür wurden zwölf Indizes auf häufig verwendete Such- und Verknüpfungsspalten angelegt. Mithilfe von EXPLAIN-Abfragen konnten die Ausführungspläne analysiert werden. Die Ergebnisse zeigen, dass PostgreSQL die definierten Indizes bei Suchanfragen und Join-Operationen effektiv nutzt. Dadurch werden Zugriffszeiten reduziert und die Skalierbarkeit der Anwendung verbessert.

3. Ergebnisse und Fazit

Das finale Schema bildet alle wesentlichen Geschäftsprozesse der Anwendung ab und erfüllt die Anforderungen an Datenintegrität, Erweiterbarkeit und Wartbarkeit.

Durch die konsequente Anwendung der dritten Normalform konnten Redundanzen weitgehend eliminiert werden. Die Einführung der separaten Autorenverwaltung stellt eine zusätzliche Verbesserung der Datenqualität dar und erhöht die Flexibilität des Modells. Die implementierten Trigger sorgen für eine automatisierte Durchsetzung wichtiger Geschäftsregeln und tragen zur Konsistenz der gespeicherten Daten bei.

Besonders hervorzuheben ist die Kombination aus strukturiertem Datenmodell, automatisierten Tests und Performanceoptimierungen. Dadurch entstand nicht nur ein theoretisch korrektes Schema, sondern eine praktisch nutzbare Datenbanklösung, die den Anforderungen einer realen Anwendung gerecht wird.

Für zukünftige Erweiterungen wären unter anderem eine Reservierungsfunktion, Benachrichtigungssysteme, eine feinere Standortsuche mit PostGIS sowie zusätzliche Auswertungsfunktionen denkbar. Die bestehende Architektur bietet hierfür bereits eine solide Grundlage.

4. GitHub Repository

Der vollständige Quellcode des Projekts ist öffentlich über GitHub verfügbar. Das Repository enthält sämtliche SQL-Skripte zur Erstellung des Datenbankschemas, die implementierten Trigger und Testfälle sowie alle Dateien der einzelnen Projektphasen.

Repository: https://github.com/ecttce/Data_Mart